Welcome to Week 2: Training Neural Networks

Last week's focus was on **inference**, which is the process of using an already-trained neural network to make predictions.

This week's focus is on **training**, which is the process of learning the optimal parameters (weights and biases) for a neural network based on a given training set.

The 3-Step Training Process in TensorFlow 🚀

We will continue using the handwritten digit recognition example (a 3-layer network: 25 units, 15 units, 1 unit). Training this model in TensorFlow involves three key steps:

1. Define the Model Architecture

- This step is identical to last week. You use `Sequential` to define the stack of layers.
- This tells TensorFlow how to perform the forward propagation (inference) calculation.

```
model = Sequential([
    Dense(units=25, activation='sigmoid'),
    Dense(units=15, activation='sigmoid'),
    Dense(units=1, activation='sigmoid')
])
```

2. Compile the Model

- This step configures the model for training.
- The most critical component you specify here is the **loss function** (or cost function) that you want the model to minimize.
- For a binary (0/1) classification problem like this, the standard loss function is **Binary Crossentropy**.

```
model.compile(loss='binary_crossentropy')
```

3. Train the Model ("Fit")

- This step actually runs the training process (i.e., gradient descent).
- You call `model.fit()` and provide your training data (`X`) and your target labels (`Y`).
- You also specify the number of **epochs**. An **epoch** is a technical term for one full pass or iteration of the learning algorithm over the entire training set.

```
model.fit(X, Y, epochs=100)
```

Why Understand "Under the Hood"?

While these three function calls are all you need to train a model, it's crucial to understand *what* they are doing. This conceptual framework is essential for **debugging** your model when it doesn't work as expected.